

PwdScope

PASSWORD STRENGTH ANALYZER & CUSTOM WORDLIST GENERATOR

Name: Jennita Angel A
Date: 13/09/2026 – 23/09/2026

1. INTRODUCTION

Passwords are widely used to protect digital accounts and personal information from unauthorized access. However, passwords that are short, predictable or based on common words and personal details can be easier to guess. Password strength depends on factors such as length, complexity and unpredictability. Understanding these factors helps users create more secure passwords.

PwdScope – Password Strength Analyzer & Custom Wordlist Generator was developed as part of my Cybersecurity internship at Elevate labs to explore password security through practical implementation. This project focuses on evaluating password strength and demonstrating how common patterns and personal details can be used to generate password variations. Additional features include password pattern checks, optional personal-context analysis and a graphical interface to make the application easier to use.

2. ABSTRACT

PwdScope combines password strength analysis with custom wordlist generation. It uses the zxcvbn library to estimate password strength and provide feedback, while additional checks identify selected patterns such as repeated characters, sequences, years, substitutions and optional personal terms.

The wordlist generator uses inputted details - including names, pet names, years and phrases to create variations through changes in letter case, selected leetspeak substitutions and combinations. Generated wordlist can be saved to a text file. The application provides both a Command-Line Interface (CLI) and a Graphical User Interface (GUI).

The project demonstrates the practical use of Python for password analysis, text processing and application development. It is intended to help users understand password predictability and support authorized security testing with fictional or permitted sample data.

3. TOOLS USED

Tool	Purpose
Python	Main programming language used to develop PwdScope.
zxcvbn	Estimates password strength and provides feedback.
NLTK	Supports text processing and tokenization of phrase inputs.
argparse	Handles command-line arguments and project commands.
Tkinter	Used to develop the graphical user interface (GUI).
pytest	Used to run automated tests on project modules.
Visual Studio Code	Code editor used for project development.

4. STEPS INVOLVED IN BUILDING THE PROJECT

- i. **Project Setup:** Set up the project in VS Code using a Python virtual environment. Installed the required libraries and organized the source code, test files and output folder into separate directories.
- ii. **Password Strength Analysis:** Integrated the `zxcvbn` library to evaluate passwords and obtain a strength score from 0 to 4. Mapped the scores to strength categories and displayed the associated warnings and suggestions.
- iii. **Password Pattern Checking:** Added separate checks to identify repeated characters, common sequences, four-digit years and selected substitution characters. Included optional personal-context inputs so supplied terms could also be checked against the password.
- iv. **Custom Wordlist Generation:** Processed the supplied name, pet name, year and phrase inputs to form candidate words. Generated variations using different letter cases, selected leetspeak substitutions, year combinations and name-pet combinations. Used a set to remove duplicate worlists and applied a maximum-item limit to the returned list.
- v. **Wordlist Export:** Added a feature to save the generated custom wordlist to a text file, with each password variation written on a separate line.
- vi. **Command-Line Interface (CLI):** Organized the application's operations into analyze and generate commands using `argparse`. Used command-line arguments to provide inputs and control wordlist generation.
- vii. **Graphical User Interface (GUI):** Developed a Tkinter interface with separate tabs for password analysis and wordlist generation. Connected the input fields and buttons to the existing project functions, displayed analysis results and added a save-file dialog for exporting wordlists.
- viii. **Testing and Verification:** Prepared test cases for the analyzer, pattern checker and wordlist generator. Ran the tests using `pytest` and checked the main CLI and GUI functions.

5. CONCLUSION

Working on *PwdScope* as part of my Cybersecurity Internship at Elevate Labs helped me understand password security while gaining practical experience in developing a security-related application. I learned how password strength can be evaluated, how common patterns and personal details affect password predictability and how custom wordlists can be generated for authorized testing. Implementing these features using Python, along with developing CLI and GUI interfaces, allowed me to use my programming skills to a cybersecurity task. Testing the project also helped me improve my problem-solving and debugging skills.

Thus, this project strengthened both my understanding of password security and my ability to develop practical tools to explore cybersecurity concepts.